

Player Performance Prediction (Group 1)

Team Github Repository

<https://github.com/orgs/Professional-Team-Project-GROUP-1/repositories>

5. Test Plan

1. Purpose and Scope

- This updated Test Plan describes the testing process for the completed “CricPredict – Player Performance Analyzer” project. The final system predicts a player’s overall performance score on a scale from 0 to 10 by using historical batting, bowling, fielding, and career statistics.
- The completed application supports three player roles: batsman, bowler, and all-rounder. The user selects a role and model, enters the required statistics, and receives a predicted score, performance category, gauge, and supporting analysis.

In Scope

Area	Test Objective
Datasets testing	Validate the IPL and BCCI datasets, required columns, numeric values, missing data, duplicates, and processed records.
Feature testing	Verify the 18 general features and the correct role-specific feature sets for batsmen, bowlers, and all-rounders.
Model testing	Validate Random Forest, MLP, XGBoost, LightGBM, Hybrid, and ensemble models included in the completed project.
Prediction testing	Verify that predictions remain within the 0–10 range and match the correct performance category.
Role-based testing	Confirm that each player role loads the correct model, metadata, and input fields.
Application testing	Test Streamlit launch, input validation, model selection, prediction display, charts, and error handling.
Documentation and code	Review coding standards, notebooks, model artifacts, README, model card, and training reports.

Out of Scope

- This test plan does not cover live ball-by-ball prediction, guaranteed match results, betting advice, real-time injury or weather updates, final playing XI confirmation, security penetration testing, or high-volume load testing.
- The completed project does not provide separate Test, ODI, and T20 match-format prediction. It uses the final combined project datasets and role-specific models.

2. Coding Standard and Code Review

- Python is the primary programming language. Pandas and NumPy are used for data processing; Scikit-learn, XGBoost, LightGBM, and joblib are used for modelling and persistence; Streamlit and Plotly are used for the application interface.
- Variables and functions use lowercase with underscores, for example: `player_role`, `prepare_input_data()`, and `load_model()`.
- Class names use PascalCase and files use descriptive lowercase names with underscores where applicable.
- The code is separated into data processing, feature engineering, model training, evaluation, saved artifacts, notebooks, and application components.
- Code review must verify feature order, role mapping, model loading, data validation, exception handling, reproducibility, and consistency between the application and model metadata.

3. Data Quality Testing and Validation

- The completed package contains IPL and BCCI cricket data, processed datasets, model-ready records, metadata files, and trained model artifacts. Data testing ensures that incorrect or incomplete values do not produce invalid predictions.

Data Validation Tests

Test ID	Test Scenario	Expected Result
TC-DATA-01	Load all required raw and processed CSV files.	Files load successfully and contain records.
TC-DATA-02	Check required batting, bowling, fielding, and career columns.	All columns required by the training pipeline are available.
TC-DATA-03	Check missing and non-numeric values.	Values are converted or handled without pipeline failure.
TC-DATA-04	Check duplicate player records.	Duplicates are removed or handled according to the data-processing logic.
TC-DATA-05	Check invalid negative statistics.	Invalid values are rejected, corrected, or prevented by application limits.
TC-DATA-06	Check processed dataset size.	The model-ready records agree with the training and evaluation reports.
TC-DATA-07	Check player names and source values against sample official records.	Selected records are consistent with the official source or documented differences are explained.

Feature and Role Validation

Scenario	Expected Result
General model selected	The application uses the full ordered set of 18 model features.
Batsman role selected	Only the batsman model and batsman-related inputs are used.
Bowler role selected	Only the bowler model and bowler-related inputs are used.
All-rounder role selected	The all-rounder model uses both batting and bowling information.
Feature order changed or missing	The system rejects the input or displays a controlled error instead of returning an unreliable prediction.

4. Unit, Integration, and Regression Testing

- The completed project contains notebooks, training code, saved models, comparison results, and application logic. However, a dedicated automated pytest test folder was not found in the supplied project package.
- The following tests should be retained as repeatable regression tests so that future changes do not break the completed system.

Test Area	Functions or Components to Test
Data utilities	CSV loading, numeric conversion, missing-value handling, duplicate removal, and processed dataset creation.
Feature engineering	Feature names, feature count, role-specific feature selection, target calculation, and feature order.
Model utilities	Model loading, metadata loading, deterministic inference, prediction clipping, and category conversion.
Application logic	Role selection, model selection, input preparation, validation, error messages, and output rendering.
Integration	Complete flow from user input to feature vector, saved model prediction, score category, and analysis display.

5. Model Performance Evaluation Against KPIs

- Regression models are evaluated using R^2 , Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE). The recorded results in the completed project show that the best models perform strongly on the held-out test data.

Model / Role	R^2	MAE	RMSE	Result
General LightGBM	0.9803	0.0771	0.1097	Best general model
Batsman Ensemble	0.9807	0.0678	0.1054	Best batsman result
Bowler Ensemble	0.9527	0.0890	0.1367	Best bowler result
All-rounder LightGBM	0.9069	0.1369	0.2486	Best all-rounder result

Model Test Scenarios

Test ID	Test Scenario	Expected Result
TC-MODEL-01	Load every saved .pkl model.	Each artifact loads without corruption or dependency error.
TC-MODEL-02	Compare model feature metadata with application inputs.	Names, number, and order of features match exactly.
TC-MODEL-03	Run the same valid input more than once.	The model returns the same prediction for identical input.
TC-MODEL-04	Test minimum and maximum permitted inputs.	The application returns a valid score between 0 and 10.
TC-MODEL-05	Recalculate evaluation metrics.	Metrics match the recorded values within an agreed tolerance.
TC-MODEL-06	Remove or rename a model artifact.	A readable error is shown and the application does not crash unexpectedly.

KPI Acceptance Criteria

- The application successfully predicts for batsman, bowler, and all-rounder roles.
- The best completed models achieve positive and strong R^2 results and improve upon simple baseline performance.
- All saved model artifacts load successfully and predictions remain in the valid 0–10 range.
- The application clearly shows the predicted score and corresponding performance category.
- Training results, saved model metadata, and application feature order remain consistent.

6. Application and User Acceptance Testing

- The Streamlit application is tested as a complete user workflow. The expected users include cricket fans, analysts, students, coaches, and project reviewers.

UAT Scenario	Expected Result
Launch the application	The Streamlit page opens without a fatal import or model-loading error.
Select batsman	The correct input form and available batsman models are displayed.
Select bowler	The correct input form and available bowler models are displayed.
Select all-rounder	Combined batting and bowling inputs are displayed.
Enter valid statistics	A score, category, gauge, and analysis are displayed.
Enter boundary values	The system handles zero and maximum permitted values without invalid output.
Invalid or incomplete artifact	A clear error or warning is displayed instead of an uncontrolled crash.
Repeat prediction	Identical input and model selection produce a consistent result.
Review charts and information tabs	Supporting charts and model information render clearly.

Example Output

Role: Bowler Model: Ensemble Predicted Performance Score: 7.8 / 10

Performance Category: Very Good

The actual score depends on the player statistics entered by the user and must be presented as an analytical prediction rather than a guaranteed future result.

UAT Pass Criteria

- A reviewer can complete at least one prediction for each role without assistance.
- The application does not crash during the normal prediction workflow.
- The score, category, and supporting information are understandable and internally consistent.
- At least 80% of reviewers rate the workflow as clear and easy to use.
- No critical or high-severity defect remains open at final sign-off.

7. Defect Management and Sign-Off Process

- Each defect should record a test ID, environment, steps to reproduce, expected result, actual result, severity, supporting screenshot or log, assigned owner, and retest status.
- After correction, the failed test and related regression tests must be repeated before the defect is closed.
- Final sign-off takes place after data validation, feature validation, model verification, application testing, UAT, documentation review, and confirmation of known limitations.

Final Sign-Off Checklist

Sign-Off Item	Status Required
Data files and required columns	Validated
General and role-specific feature order	Validated
Saved model and metadata files	Load successfully
Recorded evaluation metrics	Reproduced within tolerance
Streamlit application	Launches and predicts for all roles
Input validation and error handling	Verified
User acceptance scenarios	Passed
README, model card, and final documentation	Completed

8. Known Gaps and Recommended Improvements

- Add a dedicated tests folder using pytest for data utilities, feature order, role classification, model loading, category conversion, and deterministic inference.
- Add an automated smoke test or continuous-integration workflow that imports the application and loads every saved model artifact.
- Use a time-aware or player-grouped evaluation split in future versions to provide stronger protection against data leakage when forecasting future performance.
- Keep all documentation consistent with the final implemented models and remove references to models or functions that were planned but not included in the final application.
- Retain written evidence of sample validation against official BCCI or source records.

Response to Supervisor's Comment

Clarification: Data Validation Against Official Source

- The project data should be verified by comparing a sample of downloaded or scraped player records with the original official source. Important fields such as matches, runs, batting average, strike rate, wickets, bowling average, economy rate, and catches should be checked.
- Public datasets such as Kaggle may be used as a secondary reference for structure and general consistency, but the official source remains the main source of truth.
- Any difference caused by update dates, competition scope, or cleaning rules should be documented as part of the test evidence.